

Biomedical Signal Processing and Machine Learning

Prof. Stefano Diciotti's Module – Practical part

Second Cycle Degree/Two-Year Master's in Biomedical Engineering

8 June 2026

Setting up the development platform (Google Colab)

1. Go to eol.unibo.it, log in to today's session, and download the file 'Template_BSPML_20260608.ipynb'.
2. Go to <https://colab.research.google.com/>, and log in at the top right with your Google account.
3. Upload the template to Google Colab, for example, by selecting 'File -> Upload Notebook -> Upload'

Documentation

Directly use the following links for the Python documentation (DO NOT use Google or other search engines because of firewall settings):

- <https://pandas.pydata.org/docs/>
- <https://scikit-learn.org/stable/>
- <https://numpy.org/doc/>
- <https://matplotlib.org/stable/index.html>

Exercise: Hyperparameter optimization with nested cross-validation

You are provided with a synthetic biomedical dataset generated using `make_classification()`. The dataset contains 500 samples and 20 features, representing a binary classification problem (e.g., healthy vs. diseased patients).

Your task is to build and evaluate a machine learning model using Logistic Regression and nested cross-validation.

Requirements:

1. Use a Logistic Regression classifier.
2. Define a hyperparameter grid for the regularization parameter C with the following values:
 - 0.01
 - 0.1
 - 1
 - 10

- 100
- 3. Implement an inner 5-fold cross-validation loop to optimize hyperparameters using GridSearchCV.
- 4. Implement an outer cross-validation loop using cross_validate with 5-fold cross-validation to estimate the model's generalization performance.
- 5. Use ROC AUC as the optimization metric during hyperparameter tuning.
- 6. Evaluate the final model using the following metrics:
 - accuracy
 - ROC AUC
- 7. Report:
 - The mean and standard deviation of accuracy across the outer folds.
 - The mean and standard deviation of ROC AUC across the outer folds.

Questions (please answer in a text cell):

1. Why is nested cross-validation preferred over a single cross-validation procedure when hyperparameters must be optimized?
2. Which metric (accuracy or ROC AUC) is more informative for this biomedical classification problem? Explain your answer.

Additional analysis: effect of class imbalance

The initial dataset has a class distribution defined by:

weights=[0.6, 0.4]

Repeat the analysis with increasing levels of class imbalance:

weights=[0.8, 0.2]

weights=[0.96, 0.04]

For each dataset:

1. Perform the same nested cross-validation procedure described above.
2. Report the mean Accuracy and mean ROC AUC obtained across the outer folds.
3. Compare the results obtained for the three datasets.

Discussion Questions (please answer in a text cell)

1. How does increasing class imbalance affect accuracy and ROC AUC?
2. Which metric appears to be more robust to class imbalance?
3. In a biomedical diagnosis setting, would accuracy alone be sufficient to evaluate model performance? Why or why not?

Submitting the solution on eol.unibo.it

Save your Python code on your PC, rename it, and upload it to eol.unibo.it

DO NOT MODIFY THE NEXT CELL (PENALTY of 0.5 POINTS)

```
In [ ]: from sklearn.datasets import make_classification

# Synthetic biomedical data

X, y = make_classification(n_samples=500, n_features=20, n_informative=5,
                           n_redundant=3, weights=[0.6, 0.4], class_sep=1.0, random_state=42)
```

PLACE BELOW YOUR CODE

```
In [ ]: from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import KFold, cross_validate, train_test_split,
GridSearchCV
from sklearn.metrics import mean_absolute_error
import numpy as np

SEED = 65

outer_n_folds = 5

inner_n_folds = 5

C = [0.01, 0.1, 1, 10, 100]

outer_cv = KFold(n_splits=outer_n_folds, shuffle=True, random_state=SEED)

inner_cv = KFold(n_splits=inner_n_folds, shuffle=True, random_state=SEED)

clf = LogisticRegression(max_iter=1000)

p_grid = [{'C': C}]

clf_gs = GridSearchCV(clf, param_grid=p_grid, cv=inner_cv, refit='roc_auc',
                      scoring='roc_auc', verbose=4)

nested_score = cross_validate(clf_gs, X=X, y=y, cv=outer_cv,
                              return_train_score=True, return_estimator=True, scoring= ['accuracy', 'roc_auc'])

acc = nested_score['test_accuracy']
auc = nested_score['test_roc_auc']

print(f"Accuracy : mean = {acc.mean():.4f} std = {acc.std():.4f}")
print(f"ROC AUC : mean = {auc.mean():.4f} std = {auc.std():.4f}")

# utile per il commento: quale C è stato scelto in ogni fold esterno
for i, est in enumerate(nested_score['estimator'], 1):
    print(f"Fold {i}: C = {est.best_params_['C']}")
```

1) With a single CV the same data does double duty: it picks C and it estimates performance. That inflates the score, since you end up rewarding the C that happens to suit those particular folds. Nested CV keeps the two jobs apart. The inner loop chooses C, the outer loop scores the result on folds that played no part in the choice, so the outer average reflects how the whole procedure would behave on new data.

2) ROC AUC, because the data is imbalanced (60/40) and accuracy gives too much credit for getting the common class right. What you actually want to know is whether the model can rank patients by risk, and that is what ROC AUC measures, regardless of how common each class is.

```
In [ ]: import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_validate, GridSearchCV

SEED = 65
C = [0.01, 0.1, 1, 10, 100]
weight_settings = [[0.6, 0.4], [0.8, 0.2], [0.96, 0.04]]

results = {}

for w in weight_settings:
    X, y = make_classification(n_samples=500, n_features=20, n_informative=5,
                              n_redundant=3, weights=w, class_sep=1.0,
                              random_state=42)

    clf = LogisticRegression(max_iter=1000)
    p_grid = [{'C': C}]

    # Inner loop: 5-fold CV to tune C, optimizing ROC AUC
    inner_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
    # Outer loop: 5-fold CV to estimate generalization performance
    outer_cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

    clf_gs = GridSearchCV(clf, param_grid=p_grid, cv=inner_cv, scoring='roc_auc')
    nested = cross_validate(clf_gs, X=X, y=y, cv=outer_cv,
                            scoring=['accuracy', 'roc_auc'])

    acc, auc = nested['test_accuracy'], nested['test_roc_auc']
    results[tuple(w)] = (acc, auc)

    print(f"weights = {w}")
    print(f" mean Accuracy = {acc.mean():.4f} (std {acc.std():.4f})")
    print(f" mean ROC AUC = {auc.mean():.4f} (std {auc.std():.4f})")
    print()

# Comparison table
print("Summary")
print(f"{'weights':<14}{'mean_acc':>12}{'mean_auc':>12}")
for w, (acc, auc) in results.items():
    print(f"{str(list(w)):<14}{acc.mean():>12.4f}{auc.mean():>12.4f}")
```

1) The two metrics pull in opposite directions. As the minority class shrinks, accuracy climbs ($0.87 \rightarrow 0.90 \rightarrow 0.96$) while ROC AUC drops ($0.92 \rightarrow 0.89 \rightarrow 0.78$). The rising accuracy is an artifact: at 96/4, guessing "healthy" every time already scores about 0.954, so the model's 0.956 is barely an improvement. ROC AUC tells the real story. With only few sick patients to learn from, the model separates the classes noticeably worse.

2) ROC AUC. It is built from sensitivity and specificity, measured inside each class separately, and it averages over all decision thresholds. That is why it stays stable no matter how skewed the classes are. Accuracy has no such protection: it is a raw count of correct predictions, so the majority class dominates it as soon as the split gets lopsided.

3) No. The patients who matter most in diagnosis are usually the rare ones, and accuracy barely registers them. A 96% accurate model that quietly labels nearly everyone "healthy" misses almost every real case, which in a clinical setting means missed diagnoses. You need metrics that watch the minority class directly, like recall and F1, with the threshold set by how costly a false negative is versus a false positive.

Biomedical Signal Processing and Machine Learning – Module 1 – Practical exam (AA 2023-24)

Second Cycle Degree/Two Year Master In Biomedical Engineering

2 July 2024

Setting up the development platform (Google Colab)

1. Go to eol.unibo.it, log in to today's session, and download the file 'Template_Module1_20240702.ipynb'.
2. Go to <https://colab.research.google.com/>, and log in at the top right with your Google account.
3. Upload the template to Google Colab, for example, by selecting 'File -> Upload Notebook -> Upload'

Documentation

Directly use the following links for the Python documentation (DO NOT use Google or other search engines because of firewall settings):

- <https://pandas.pydata.org/docs/>
- <https://scikit-learn.org/stable/>
- <https://numpy.org/doc/>
- <https://matplotlib.org/stable/index.html>

Exercise

In this exercise, you will explore the performance of a linear regression model using repeated 5-fold cross-validation (CV) on synthetic biomedical data. The goal is to understand how the choice of training and validation splits affects model performance.

Steps to Follow:

1. **Generate Synthetic Data:**
 - Use the provided code in the template to generate synthetic biomedical data with 200 samples and 10 features. The target variable (y) represents a linear combination of features with added noise.
2. **Implement Cross-Validation:**
 - Using Python and scikit-learn, implement a loop that performs repeated 5-fold cross-validation (n_repeats = 10) with a LinearRegression model.
 - For each repeat and fold, compute the Mean Absolute Error (MAE) on the test set.

3. Calculate and plot results:

- Calculate the average MAE across all folds for each repeat.
- Plot a line graph where the x-axis represents the iteration number (1 to 10) and the y-axis represents the average MAE.

4. Analysis:

- Why is it important to repeat cross-validation multiple times? Discuss the variability in MAE observed across different iterations of cross-validation (write your answer in a Google Colab text cell).

Submitting the solution on eol.unibo.it

Save your Python code on your PC, rename it, and upload it to eol.unibo.it

Do not modify the next code cell

```
In [ ]: import numpy as np

# Generate Synthetic Data
np.random.seed(42)
n_samples = 200
n_features = 10

# Features are random values
X = np.random.rand(n_samples, n_features)

# Target variable is a linear combination of features with some noise
true_coefficients = np.random.rand(n_features)
y = X @ true_coefficients + np.random.normal(scale=0.5, size=n_samples)
```

Place below your code

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import RepeatedKFold, cross_validate

SEED = 42
n_folds = 5
n_repeats = 10

reg = LinearRegression()
cv = RepeatedKFold(n_splits=n_folds, n_repeats=n_repeats, random_state=SEED)

score = cross_validate(reg, X=X, y=y, cv=cv, scoring='neg_mean_absolute_error')

# 50 individual MAE values (neg -> positive)
test_mae = np.abs(score['test_score'])

# Reshape to (n_repeats, n_folds) = (10, 5): each ROW is one repeat
mae_per_fold = test_mae.reshape(n_repeats, n_folds)

# Average across the 5 folds -> one value per repeat (10 values)
avg_mae_per_repeat = mae_per_fold.mean(axis=1)

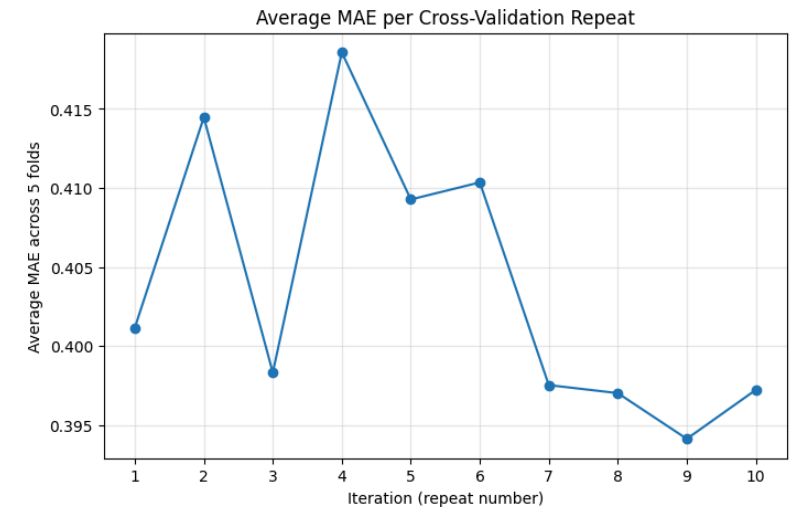
for i, m in enumerate(avg_mae_per_repeat, start=1):
    print(f"Repeat {i:2d}: average MAE = {m:.4f}")

# --- Plot ---
iterations = np.arange(1, n_repeats + 1)  # x-axis: 1 to 10

plt.figure(figsize=(8, 5))
plt.plot(iterations, avg_mae_per_repeat, marker='o', linestyle='-')
plt.title('Average MAE per Cross-Validation Repeat')
plt.xlabel('Iteration (repeat number)')
plt.ylabel('Average MAE across 5 folds')
plt.xticks(iterations)
```

```
plt.grid(True, alpha=0.3)
plt.show()
```

```
Repeat 1: average MAE = 0.4012
Repeat 2: average MAE = 0.4145
Repeat 3: average MAE = 0.3983
Repeat 4: average MAE = 0.4186
Repeat 5: average MAE = 0.4093
Repeat 6: average MAE = 0.4104
Repeat 7: average MAE = 0.3975
Repeat 8: average MAE = 0.3970
Repeat 9: average MAE = 0.3941
Repeat 10: average MAE = 0.3972
```



A single 5-fold cross-validation splits the data into folds only once. Because that split is random, the resulting MAE depends on which samples happen to land in each fold. With a small dataset (200 samples), one "lucky" or "unlucky" split can give a misleadingly low or high error — the estimate is sensitive to the particular partitioning. Repeating the cross-validation 10 times (each repeat reshuffling the data before making new folds) produces 10 different estimates of the average MAE. This lets us:

Reduce the variance of the performance estimate by averaging over many splits, giving a more reliable measure of how the model generalizes.

Quantify the uncertainty — the spread (standard deviation) of the per-repeat MAE values tells us how stable the model's performance is with respect to the choice of train/validation split.

Biomedical Signal Processing and Machine Learning – Module 1 – Practical exam

Second Cycle Degree/Two Year Master In Biomedical Engineering

7 July 2025

Setting up the development platform (Google Colab)

1. Go to eol.unibo.it, log in to today's session, and download the file 'Template_Module1_20250704.ipynb'.
2. Go to <https://colab.research.google.com/>, and log in at the top right with your Google account.
3. Upload the template to Google Colab, for example, by selecting 'File -> Upload Notebook -> Upload'

Documentation

Directly use the following links for the Python documentation (DO NOT use Google or other search engines because of firewall settings):

- <https://pandas.pydata.org/docs/>
- <https://scikit-learn.org/stable/>
- <https://numpy.org/doc/>
- <https://matplotlib.org/stable/index.html>

Exercise

In this exercise, you will evaluate a logistic regression model on an imbalanced binary classification problem, using two different performance metrics: accuracy and AUC (Area Under the ROC Curve):

1. **Data generation**
 - The dataset will be provided in the code template. It consists of 1,000 samples and 20 numerical features, generated using `make_classification`. The data is strongly imbalanced, with 95% of samples in class 0 (e.g., healthy) and 5% in class 1 (e.g., disease).
2. **Model training**
 - Train a **logistic regression** model (`LogisticRegression`, from `sklearn.linear_model`) using `C = 1` as the regularization parameter.
 - Use **5-fold cross-validation** to evaluate model performance.
3. **Evaluation**
 - Compute both **accuracy** and **AUC** on the test set of each fold.
 - Report the **mean** and **standard deviation** of each metric across the folds.
4. **Written discussion (in a Google Colab text cell):**
 - Does accuracy reflect how well the model identifies the minority class?
 - In a clinical setting, which metric would be more appropriate and why?

Submitting the solution on eol.unibo.it

Save your Python code on your PC, rename it, and upload it to eol.unibo.it

DO NOT MODIFY THE NEXT CELL (PENALTY of 0.5 POINTS)

```
In [ ]: from sklearn.datasets import make_classification

# Step 1: Generate synthetic imbalanced dataset
X, y = make_classification(
    n_samples=1000,
    n_features=20,
    n_informative=15,
    n_redundant=5,
    n_clusters_per_class=2,
    weights=[0.95, 0.05], # 95% class 0, 5% class 1
    random_state=42
)
```

Place below your code

```
In [ ]: from sklearn.model_selection import cross_validate, StratifiedKFold
from sklearn.linear_model import LogisticRegression
import numpy as np

SEED = 42
n_folds = 5

clf = LogisticRegression(C=1, max_iter=1000)

# StratifiedKFold preserves the 95/5 class ratio in every fold
cv = StratifiedKFold(n_splits=n_folds, shuffle=True, random_state=SEED)

score = cross_validate(
    clf, X=X, y=y, cv=cv,
    scoring=['accuracy', 'roc_auc'],
    return_train_score=True,
)

acc = score['test_accuracy']
auc = score['test_roc_auc']

print("\nPer-fold Accuracy:", np.round(acc, 4))
print("Per-fold AUC:      ", np.round(auc, 4))

print(f"\nAccuracy: mean = {acc.mean():.4f}, std = {acc.std():.4f}")
print(f"AUC:      mean = {auc.mean():.4f}, std = {auc.std():.4f}")
```

```
Per-fold Accuracy: [0.955 0.935 0.955 0.95 0.97 ]
Per-fold AUC:      [0.7216 0.8    0.887 0.748 0.8879]
```

```
Accuracy: mean = 0.9530, std = 0.0112
AUC:      mean = 0.8089, std = 0.0689
```

Biomedical Signal Processing and Machine Learning – Module 1 – Practical exam

Second Cycle Degree/Two Year Master In Biomedical Engineering

21 July 2025

Setting up the development platform (Google Colab)

1. Go to eol.unibo.it, log in to today's session, and download the file 'Template_Module1_20250721.ipynb'.
2. Go to <https://colab.research.google.com/>, and log in at the top right with your Google account.
3. Upload the template to Google Colab, for example, by selecting 'File -> Upload Notebook -> Upload'

Documentation

Directly use the following links for the Python documentation (DO NOT use Google or other search engines because of firewall settings):

- <https://pandas.pydata.org/docs/>
- <https://scikit-learn.org/stable/>
- <https://numpy.org/doc/>
- <https://matplotlib.org/stable/index.html>

Exercise

You are provided with a dataset generated from an unknown function with added noise. The data is already loaded in the variables X and y in the provided Python template file.

1. Split the dataset into training and test sets.
2. For each polynomial degree from 1 to 10:
 - Train the polynomial model on the training data.
 - Compute the Mean Squared Error (MSE) on both the training and test sets.
3. Create a plot showing training MSE and test MSE as functions of the polynomial degree.
4. Discuss the results

Submitting the solution on eol.unibo.it

Save your Python code on your PC, rename it, and upload it to eol.unibo.it

DO NOT MODIFY THE NEXT CELL (PENALTY of 0.5 POINTS)

```
In [ ]: import numpy as np

np.random.seed(0)
n_samples = 100
X = np.linspace(-3, 3, n_samples).reshape(-1, 1)

def true_function(x):
    return 0.5 * x**4 - x**3 - x**2 + x + 2

y_true = true_function(X)
noise = np.random.normal(0, 5, size=y_true.shape)
y = y_true + noise

# DO NOT MODIFY THIS CODE CELL (PENALTY of 0.5 POINTS)
```

Place below your code

```
In [ ]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt

# -----
# 1. Single train / test split
# -----
SEED = 42
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED
)

# -----
# 2. For each polynomial degree 1..10: fit and record BOTH MSEs
# -----
degree_list = range(1, 11)
train_mse_list = []
test_mse_list = []

for d in degree_list:
    poly = PolynomialFeatures(degree=d)
    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)      # transform only (use train's fit)

    reg = LinearRegression().fit(X_train_poly, y_train)

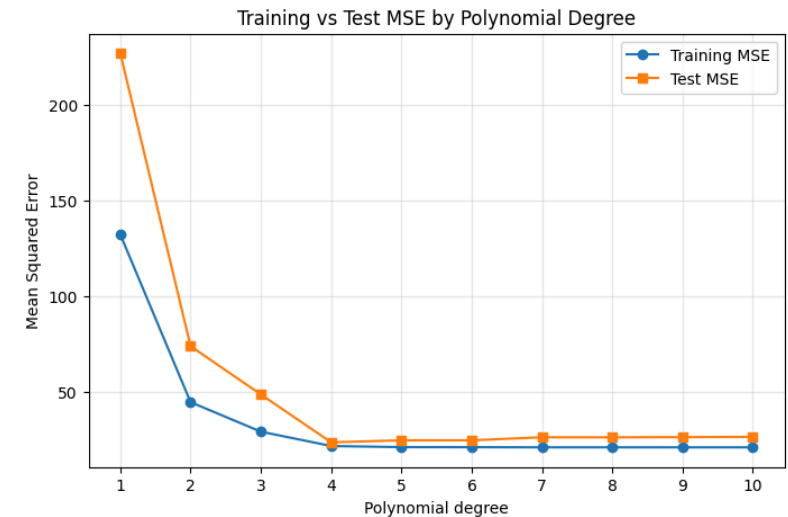
    train_mse = mean_squared_error(y_train, reg.predict(X_train_poly))
    test_mse = mean_squared_error(y_test, reg.predict(X_test_poly))

    train_mse_list.append(train_mse)
    test_mse_list.append(test_mse)
```

```
print(f"Degree {d:2d} | Train MSE = {train_mse:8.3f} | Test MSE = {test_mse:8.3f}")
```

```
# -----
# 3. Plot training vs test MSE as a function of degree
# -----
plt.figure(figsize=(8, 5))
plt.plot(list(degree_list), train_mse_list, marker='o', label='Training MSE')
plt.plot(list(degree_list), test_mse_list, marker='s', label='Test MSE')
plt.xlabel('Polynomial degree')
plt.ylabel('Mean Squared Error')
plt.title('Training vs Test MSE by Polynomial Degree')
plt.xticks(list(degree_list))
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

Degree 1		Train MSE =	132.213		Test MSE =	226.858
Degree 2		Train MSE =	44.543		Test MSE =	73.807
Degree 3		Train MSE =	29.134		Test MSE =	48.584
Degree 4		Train MSE =	21.634		Test MSE =	23.566
Degree 5		Train MSE =	21.035		Test MSE =	24.601
Degree 6		Train MSE =	21.026		Test MSE =	24.611
Degree 7		Train MSE =	20.950		Test MSE =	26.192
Degree 8		Train MSE =	20.949		Test MSE =	26.189
Degree 9		Train MSE =	20.940		Test MSE =	26.268
Degree 10		Train MSE =	20.939		Test MSE =	26.430



Biomedical Signal Processing and Machine Learning

Module 1 – Practical exam (AY 2023-24)

Second Cycle Degree/Two Year Master In Biomedical Engineering

9 September 2024

Setting up the development platform (Google Colab)

1. Go to eol.unibo.it, log in to today's session, and download the file 'Template_Module1_20240909.ipynb'.
2. Go to <https://colab.research.google.com/>, and log in at the top right with your Google account.
3. Upload the template to Google Colab, for example, by selecting 'File -> Upload Notebook -> Upload'

Documentation

Directly use the following links for the Python documentation (DO NOT use Google or other search engines because of firewall settings):

- <https://docs.python.org/3/>
- <https://pandas.pydata.org/docs/>
- <https://scikit-learn.org/stable/>
- <https://numpy.org/doc/>
- <https://matplotlib.org/stable/index.html>

Exercise

The goal of this exercise is to understand the concepts of overfitting and underfitting in the context of polynomial regression. You will start with the provided synthetic regression data and fit it using polynomial models of increasing complexity. You will then visualize how the model's performance changes on the training set and test set as the polynomial degree increases.

Steps to Follow:

1. **Generate Synthetic Data:**
 - The synthetic dataset has already been generated for you using a sinusoidal function ($\sin(x)$) with added Gaussian noise.
2. **Adoption of hold-out strategy and implementation of polynomial regression:**
 - Using the scikit-learn package, adopt a hold-out strategy (test set size = 30%).
 - To fit a polynomial regression model, you need to transform the input features into polynomial features. For this, use the PolynomialFeatures class from the sklearn.preprocessing module. Follow this code example to fit a polynomial regression model:

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Example: Fitting a Polynomial Regression Model
```

```

# Choose the polynomial degree
degree = 3 # You can change this to explore different degrees

# Create polynomial features
poly_features = PolynomialFeatures(degree=degree)

# Transform the training data into polynomial features
X_train_poly = poly_features.fit_transform(X_train)

# Fit the linear regression model to the transformed features
model = LinearRegression()
model.fit(X_train_poly, y_train)

# Transform the test data using the same polynomial transformation
X_test_poly = poly_features.transform(X_test)

# Predict using the model
y_train_pred = model.predict(X_train_poly)
y_test_pred = model.predict(X_test_poly)

```

- Fit polynomial regression models for degrees ranging from 1 to 15.
- For each degree:
 - Calculate the mean squared error (MSE) on both the training set and the test set.
 - Store these errors for visualization.

3. Visualize the results:

- Plot the mean squared error (MSE) for both the training set and the test set as a function of the polynomial degree.

4. Interpret the resulting graph:

- Explain how the graph illustrates the concepts of overfitting and underfitting (write your answer in a separate Google Colab text cell).

Submitting the solution on eol.unibo.it

Save your Python code on your PC, rename it, and upload it to eol.unibo.it